

Теоретические основы информатики

Разделы	Темы
Информация и данные	Основные понятия информатики Кодирование информации
Двоичная математика	Системы счисления Битовые операции
Модели и структуры данных	Типы данных Структуры данных
Хранение и обработка данных	Структурированные файлы Парсинг данных Регулярные выражения
Методы информатики	Алгоритмизация Сортировка

Беляков Андрей Юрьевич

к.т.н., доцент

Тема 8

gitverse.ru/band/TOI

Раздел 4. Хранение и обработка данных

Тема 7

Плоские и иерархические данные

- Структурированные файлы форматов json, csv, yaml, xml.
- Библиотеки и методы работы со структурированными данными.

Тема 8

Парсинг данных

- Автоматизация сбора, фильтрации и агрегации данных.
- Автоматизация обработки данных (запись, чтение, библиотеки).

Тема 9

Регулярные выражения

- Понятие, области применения, синтаксис.
- Практические примеры поиска и замены в строках.

Тема 8

Парсинг данных

Вопрос 1: Введение в автоматизацию сбора данных
requests, html, ETL

Вопрос 2: Сбор и обработка данных
статические и динамические страницы
библиотеки
форматы хранения

Вопрос 1

Введение в автоматизацию сбора данных



Цикл обработки ETL

Коротко о разметке html

Как делать запросы и получать сырые данные

парсинг (коротко) - извлечение структурированных данных

Цикл обработки ETL

Проблема:

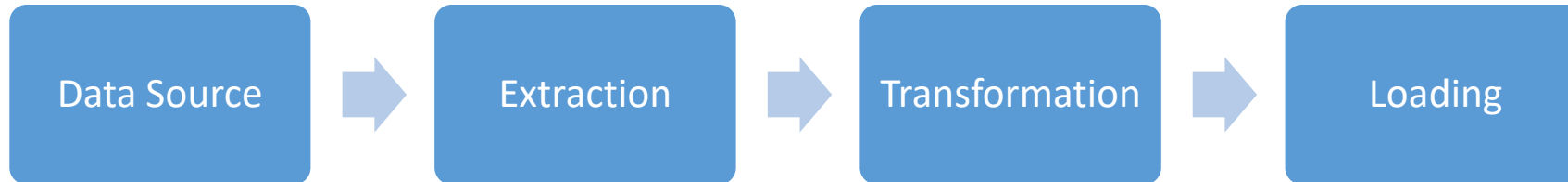
- данных слишком много
- могут быть на разных ресурсах
- могут быть не структурированы
- часто избыточны

Решение:

- автоматизация сбора и обработки данных позволяет оперативно и недорого получать отфильтрованные/обработанные данные и структурировать/агрегировать их в требуемые модели данных

Примеры:

- сбор данных по соцсетям
- сбор данных с тематических сайтов
- мониторинг цен конкурентов
- поиск и фильтрация товаров
- анализ отзывов о продукте
- сбор данных для научных исследований
- получение курсов валют/криптовалют



Data Source - html, log, txt, json, csv, docx

Extraction - парсинг/API

Transformation - очистка/фильтрация/агрегация

Loading - csv/json/бд

Обобщённый алгоритм
ETL-процесс - Extract, Transform, Load

web-parsing

- если нет официального API (json), то собираем данные с html-страниц
- если статические - requests + bs4, если динамические - Selenium

?

Этика:

- проверь robots.txt сайта, например, <https://site.com/robots.txt>
он указывает, какие страницы можно сканировать
- в User-Agent указывай, кто ты, не притворяйся браузером без необходимости
- не перегружай сайт запросами
при разработке скачай страницу и локально тренируйся
при эксплуатации используй `time.sleep()` между запросами, чтобы не создавать DoS-нагрузку
- изучай условия использования сайта
не все данные разрешено собирать и использовать в коммерческих целях

!

- DoS - Denial of Service (отказ в обслуживании) - DDoS - Distributed DoS (распределённый отказ в обслуживании)
скрипт для парсинга может создать DoS-эффект на сайт, если слишком частые запросы с одного IP-адреса
сайт может заблокировать такой IP (код 403)

Data Source

html

```
<body>
  <div class="container">
    <h1>Рейтинг абитуриентов 2025</h1>
    <table id="table-rating">
      <thead>
        <tr>
          <th>№</th>
          <th>Фамилия</th>
          <th>ЕГЭ по информатике</th>
        </tr>
      </thead>
      <tbody>
        <tr>
          <td>1</td>
          <td class="surname">Иванов</td>
          <td>94</td>
        </tr>
        <tr>
          <td>2</td>
          <td class="surname">Петров</td>
          <td>88</td>
        </tr>
      </tbody>
    </table>
  </div>
</body>
```

в html-документе

- известный формат хранения,
- иерархические теги,
- теги парные (чаще всего),
- можно искать по id, по классу, по названию тега, просто по порядку следования, по атрибуту

requests

html

User-Agent - посмотреть в браузере

```
import requests # pip install requests
```

```
url = "http://perm.1gb.ru/parsing/rating.html"
```

```
userAgent = "Chrome/151.0.0.0"
```

```
userAgent = "MyBot/1.0 (https://my-site.com; my-email@ya.ru)"
```

```
headers = { "User-Agent": userAgent }
```

```
response = requests.get(url, headers=headers)
```

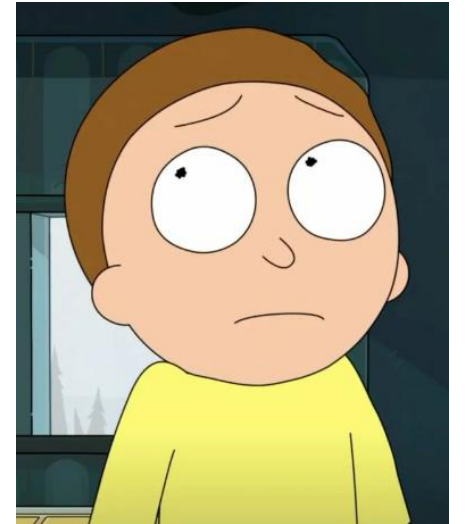
```
response.encoding = "utf8"
```

сохранить для дальнейшей
работы над парсером

```
with open("./data/rating.html", "w", encoding="utf8") as f:  
    f.write(response.text)
```


Контрольные вопросы

- этапы обработки ETL
- базовые особенности разметки html
 - теги, классы, атрибуты, id, структура DOM
- базовые принципы этики парсинга
- как делать запросы и что делать с полученной страницей
- зачем нужен User-Agent



Вопрос 2

Сбор и обработка данных



- на входе имеем структурированную строку - `html`
- есть теги (как скобки над данными)
- есть опознавательные признаки
 - классы, `id`, атрибуты, порядок следования
- можно выбирать по фильтру, сортировать
- можно формировать свои объекты
- можно агрегировать данные
- можно сохранять в удобном формате:
 - `csv`, `json`, `html`, `txt`, БД

1 - while

parsing

только фамилии

```
start, end = '<td class="surname">', '</td>'
ln = len('<td class="surname">')

with open('./data/rating.html', 'r', encoding="utf8") as f:
    s = f.read()

pos = 0
while s.find(start, pos) > -1:
    pos = s.find(start, pos)
    print(pos, s[pos+ln:pos+ln+12])
    # дописать код для извлечения данных
    pos += ln + 12
```

2 - re

parsing

только фамилии

```
# <td class="surname">Иванов</td>
import re

start, end = '<small>', '</small>'
ln = len('<small>24.03.2026</small>')
ptn = r'<td class="surname">\s*(\S+)\s*</td>'

with open('./data/rating.html', 'r', encoding="utf8") as f:
    s = f.read()
    for i, match in enumerate(re.findall(ptn, s), start=1):
        print(i, match)
```

2 - re

parsing

только фамилии

```
import re

# компилируем регулярное выражение один раз
pattern = re.compile(r'<td class="surname">\s*(\S+)\s*</td>')

with open('./data/rating.html', 'r', encoding='utf-8') as f:
    s = f.read()
    for i, match in enumerate(pattern.findall(s), start=1):
        print(i, match)
```

но тут один раз

re.compile() - предварительно откомпилированная регулярка
работает быстрее при многократном использовании

3 - bs4

parsing

ТОЛЬКО фамилии

```
from bs4 import BeautifulSoup # pip install bs4
```

```
with open('./data/rating.html', 'r', encoding='utf-8') as f:  
    soup = BeautifulSoup(f.read(), 'html.parser')
```

```
# 1 - найти все ячейки с фамилиями
```

```
surnames = soup.find_all('td', class_='surname') # 1
```

```
surnames = soup.select('td.surname') # 2
```

```
surnames = soup.select('.surname') # 3
```

```
surnames = soup.find_all('td', attrs={'class': 'surname'}) # 4
```

```
# 2 - достать все фамилии
```

```
surnames = [tag.get_text(strip=True) for tag in surnames] # 1
```

```
surnames = [tag.get_text().strip() for tag in surnames] # 2
```

```
surnames = [tag.text.strip() for tag in surnames] # ~ .get_text() # 3
```



parsing

технологии анализа html, xml

Парсер	Преимущества	Недостатки	Когда использовать
<code>html.parser</code>	встроен в Python не требует установки	менее снисходителен к плохому HTML-коду	простые и правильные HTML-страницы
<code>lxml</code>	очень быстрый работает в "грязном" HTML	требуется установки внешней библиотеки	работа с большими объемами данных и сложными страницами
<code>html5lib</code>	максимально снисходительный к нарушениям	очень медленный, требуется установки внешней библиотеки	работа с сильно сломанным или невалидным HTML
<code>lxml-xml</code>	быстрый, для парсинга XML-документов	требуется установки внешней библиотеки	парсинг XML-файлов



parsing

технологии **извлечения** сырых данных

Парсер	Преимущества	Недостатки	Когда использовать
циклы, условия, срезы	встроен в Python, можно настроить гибко	много работы для настройки поиска и выборки	простые HTML-страницы
Re	встроен в Python, можно настроить гибко	трудно для начинающих, много работы для настройки поиска и выборки	работа с большими объемами плохо структурированных данных
bs4	лёгкий старт, для обычных задачи интуитивно понятен, много настроек	требуется установки внешней библиотеки, работает только на статичных страницах	работа со сложными страницами, работа со статическими страницами
Selenium	интуитивно понятен, много настроек, есть возможность управлять событиями в браузере	требуется установки внешней библиотеки, работает медленно, требует запуска браузера	работа со сложными страницами, работа с динамическими страницами



parsing

технологии **обработки** данных

Парсер	Когда использовать
pandas	для анализа табличных данных: фильтрация, группировка, агрегация, простые математические операции
NumPy	для интенсивных численных расчетов над массивами, когда важна производительность и память
SciPy	для сложных научных и инженерных вычислений: оптимизация, интегралы, обработка сигналов
Polars	для больших наборов данных и сложных операций, где важна производительность
циклы, условия, срезы	несложные операции извлечения, небольшие объёмы, нет требований по производительности

Практика подготовки данных

html

Решим задачи сбора табличных данных из нескольких колонок

Обсудим:

- вопросы фильтрации по параметрам
 - вопросы сортировки по 1,2,3-м полям
 - вопросы агрегации, смены полей
 - вопросы сохранения – json, csv
-

Рассмотрим структуру html-страниц для:

- погодного сайта
- сайта с результатами соревнований
- сайта с рейтингом языков программирования

Обсудим возможные способы сбора данных и задания для практики

Контрольные вопросы

Особенности извлечения данных с помощью:

- цикла, re, bs4

Технологии анализа документов html, xml

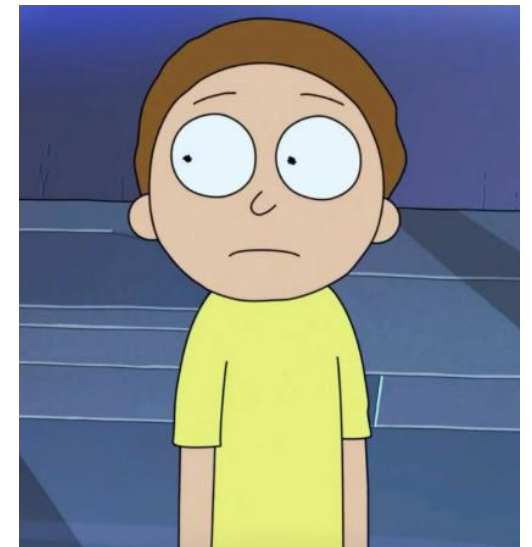
Технологии извлечения сырых данных

Технологии обработки данных

Форматы сохранения данных

Агрегация данных

Сортировка по нескольким параметрам



Теоретические основы информатики

Раздел 4. Хранение и обработка данных

Тема 8: Парсинг данных

2026

Беляков Андрей Юрьевич